Haunted Toy Room — teacher evaluation questions and answers

Prepared against the current source on 11 October 2026. Use the answers as speaking notes, then open the cited function and demonstrate the result. Source line numbers describe this revision; function names remain the useful search keys after later edits.

The current story is arrival → 257 puzzle → toy chest → Buzz's bedroom → wardrobe rescue → laser escape → morning → free exploration. Describe this implementation during evaluation. Earlier plans involving a lost RC car, rescue switches, an energy barrier, or a terminal WIN enum do not describe the current state machine.

Quick facts to memorise

Topic	Current implementation
Primitive families	5: plane, cube, sphere, cylinder, cone
Raster shading choices	4: Flat, Gouraud, Phong, Blinn-Phong
Light types	3: directional, point, spot
Light slots	8: sky, bulb, lamp spot, two headlights, laser glow, area light, ghost/story glow; enabled count varies
Material illumination terms	Ambient, diffuse, specular, plus optional emissive colour
Rendering paths	Indexed triangle rasterisation and analytic GPU ray tracing
Surface texture array	27 named layers; white fallback and HUD font atlases are additional resources
Camera modes	3: Free, Orbit, Follow; story shots temporarily position the existing camera
Story states	8: PROLOGUE, PUZZLE, TOY_CHEST, BUZZ_ROOM, WARDROBE, FINAL_ESCAPE, MORNING, FREE_EXPLORE
Hallway puzzle	3 persistent clues: train 2, clock 5, blocks 7; inspect all three before submitting 257
Stairs	18 treads; rise 4.5/18 = 0.25, run 8/18 ≈ 0.4444 per tread
Physics time step	1/120 second, at most 6 steps accumulated per frame
Ray defaults	Half window resolution; 2 secondary bounces; UI range 0–4 bounces
BVH leaf capacity	At most 2 instances
Interaction count	No single interaction enum/count: distinguish selection, movement, proximity actions, special actions, camera, editing, rendering, and world controls

Counts are defined in src/geometry/Mesh.h:15 (enum class PrimitiveType), src/render/RenderSettings.h:6 (enum class ShadingMode), src/scene/Light.h:8 (enum class LightType), src/render/Assets.h:19 (enum TextureSlot), src/world/StoryDirector.h:40 (enum class GameplayState), and src/render/RayTracer.h:39 (LeafSize).

Report sections and where to prove them

Read alongside Project-Report.md and Project-Report.pdf.

Report section	Source to open	Demonstration
2.1 Indexed geometry	Primitives.cpp, Mesh.cpp, Vertex.h	Shift+V dumps vertex/index tables
2.2 Homogeneous transformations	Transform3D.cpp, Transform.h	Tab inspector: translate, rotate, scale, shear, mirror
2.3 Hierarchy and normals	SceneNode.cpp, MatrixStack.h, normalMatrix	R mounting; nonuniform scale with correct highlights
2.4 Camera/projection/picking	Camera.cpp, ToyRoomApp::ViewRay/Pick	C, F, mouse selection
2.5 Illumination	shaders/lighting.glsl, Light.h	F5/F6/F7 terms, lamp controls
2.6 Shading	lit.frag, gouraud.vert, RenderSettings.h	F2 on the same curved surface
2.7 Texture mapping	ProceduralTextures.cpp, BmpLoader.cpp, Texture.cpp	F3; show clock face and fabric
2.8 Shadows/rays	Renderer.cpp, Ray.cpp, raytrace.frag	F4; compare floor reflections and lamp shadow
2.9 BVH/bounds	RayTracer::Build, unitBoundsRadius, PhysicsWorld::ShapeBounds	Explain conservative transformed bounds
3.2–3.6 Characters	Humanoid.cpp, Bullseye.cpp, RCCar.cpp, Cat.cpp, PennyArrival.cpp	Motion, mounting, Buzz flight and laser
3.7–3.12 Environment/story props	Room.cpp, House.cpp, StoryProps.cpp, HallwayPuzzle.cpp	Walk from entrance through stairs to clues
4.1–4.6 Visual results	UpdateLights, shared shaders, Environment.cpp	Matched camera/time/settings comparison
4.7–4.8 Collision/camera	PhysicsWorld.cpp, StepScene	Stair ascent/descent and camera obstruction
4.9 Following	CompanionFollow.cpp, StoryDirector::Navigate	Short taps versus sustained walking
4.10 Day/night	ToggleDayNight, FreezeToys, UpdateSkyHour	Slash or HUD switch, preserve stage/progress
4.11 Optimisation	mesh LOD, cached transforms, BVH, reduced ray target	Explain quality/performance tradeoffs
4.12 Verification	tests/PhysicsChecks.cpp, tools/check_escape_rendering.py	Run checks and full story replay

The report's light table names the area slot “Hall night light”; the current UpdateLights also repositions it as the corridor lamp, bedroom lamp, or porch lantern. Historical screenshots and numerical measurements are observations from their capture revision, not guaranteed results on every computer.

A. Architecture and project overview

1. Explain your project in one minute.

It is a C++/OpenGL interactive house built from reusable mathematical primitives. Penny explores at night, discovers 257, wakes the chest toys, reaches Buzz's wardrobe, and escapes through a door hit by Buzz's laser. A hierarchical scene graph supports animation and mounting; raster and analytic ray-traced rendering share scene objects, materials, lights, and textures. The game finishes its story with a morning shot and continued exploration.

Code: src/app/ToyRoomApp.cpp:77 (void ToyRoomApp::BuildScene), src/world/StoryDirector.cpp:492 (void StoryDirector::Update). Report: 1.1–1.4.

2. Where is the main application loop?

Application::Run polls events, computes/clamps delta time, updates, renders, ends the input frame and swaps buffers. ToyRoomApp supplies the game-specific update/render callbacks. Updating and drawing are separate responsibilities.

Code: src/core/Application.cpp:48 (void Application::Run), src/app/ToyRoomApp.cpp:213 (void ToyRoomApp::OnUpdate), src/app/ToyRoomApp.cpp:1417 (void ToyRoomApp::OnRender).

3. What did you implement, and what comes from libraries?

The project contains its own primitive generation, transformation formulas, scene graph, GLSL illumination, analytic ray intersections, BVH construction, procedural images, BMP reader, gameplay and collision logic. OpenGL executes rendering; GLFW handles windows/events; GLAD loads OpenGL functions; GLM supplies vector/matrix storage and operations. Do not claim to have written OpenGL, GLFW, GLAD, or GLM.

Code: src/geometry/Primitives.cpp:100 (MeshData Sphere), src/math/Transform3D.cpp:24 (glm::mat4 translate), src/gl/glad.c:152 (gladLoadGLLoader), src/core/Application.cpp:12 (glfwInit). Report: 3.1.

4. Who owns GPU resources, and why reuse primitive meshes?

Assets owns meshes, texture objects and materials. Nodes retain pointers to these shared resources. Many heads/eyes use the same sphere VBO rather than allocating a distinct mesh for each part; each instance has its own transform/material.

Code: src/render/Assets.h:16 (class Assets), src/render/Assets.cpp:14 (void Assets::Load), src/scene/SceneNode.h:31 (const Mesh* mesh).

5. What is a scene graph?

It is a tree of nodes with local transforms. Empty nodes are joints/groupings; nodes with meshes draw shapes. A depth-first update computes world = parentWorld × localMatrix. Moving a parent moves all descendants, which is essential for limbs, lamps, doors and Jessie's saddle attachment.

Code: src/scene/SceneNode.cpp:44 (void SceneNode::UpdateWorld), src/scene/MatrixStack.h:15 (class MatrixStack). Report: 2.3.

6. Is rendering responsible for gameplay or physics?

No. StepScene updates controllers, story, animation, physics and world transforms. OnRender collects the resulting visible scene and chooses the rendering path. Rendering a door differently does not itself unlock it; puzzle and story flags do that.

Code: src/app/ToyRoomApp.cpp:220 (void ToyRoomApp::StepScene), src/app/ToyRoomApp.cpp:1417 (void ToyRoomApp::OnRender).

7. How can the same scene appear in both renderers?

Renderer::CollectNode produces DrawItems with primitive mesh, model matrix, normal matrix and material. Rasterisation draws indexed triangles. RayTracer::Render uses the same items as transformed analytic instances. Hidden parents suppress their whole subtree in both paths.

Code: src/render/Renderer.cpp:221 (void Renderer::CollectNode), src/render/RayTracer.cpp:99 (void RayTracer::Render).

B. Object creation and geometry equations

8. Where are the actual object-creation equations defined?

Reusable mesh equations are in Primitives.cpp. Room/house/character constructors assemble those meshes using positions, rotations and sizes. For example, a head is a transformed sphere, not a separate imported head equation. Open the primitive generator first, then the relevant object's AddShape calls.

Code: src/geometry/Primitives.cpp:20 (namespace Primitives), src/characters/Humanoid.cpp:68 (Humanoid::Humanoid), src/world/Room.cpp:36 (RoomRig BuildRoom), src/world/House.cpp:109 (HouseRig BuildHouse). Report: 2.1 and Appendix A.

9. What does each vertex store?

A position, a normal and a UV coordinate. Positions describe geometry, normals describe surface orientation for lighting, and UVs locate image samples. An index buffer groups references into triangles.

Code: src/geometry/Vertex.h:10 (struct Vertex), src/geometry/Mesh.cpp:6 (Mesh::Mesh).

10. How is the plane generated?

It has four vertices at x,z = ±0.5, y = 0, normal (0,1,0) and rectangular UVs. Two triangles cover the square. A model transform changes its size and orientation. One-sided planes remain useful for walls; walkable floors now use thin cubes with undersides.

Code: src/geometry/Primitives.cpp:31 (MeshData Plane), src/world/Room.cpp:72 ("HallFloor").

11. Why does a cube have 24 vertices if it only has 8 corners?

Each of six faces needs four vertex records because adjoining faces need different normals and UV coordinates. A shared position does not imply identical vertex attributes. There are 12 triangles and 36 indices.

Code: src/geometry/Primitives.cpp:53 (MeshData Cube).

12. Derive the sphere vertices.

For radius r=0.5, latitude φ and longitude θ, x=r cosφ sinθ, y=r sinφ, z=r cosφ cosθ. Rings step through latitude and columns through longitude. The normal is p/r; UVs are normalised ring/column parameters. The seam column is duplicated so U can be both 0 and 1.

Code: src/geometry/Primitives.cpp:100 (MeshData Sphere). Report: 2.1.

13. How does a sphere become an ellipsoid?

Nonuniform model scaling changes a sphere into an ellipsoid: $(x/a)^2+(y/b)^2+(z/c)^2=1$ about its transformed centre. Penny's torso, eyes and muzzle use differently scaled spheres. The mesh generator remains shared; normals use the inverse transpose.

Code: src/characters/Cat.cpp:40 ("Torso"), src/math/Transform3D.cpp:148 (glm::mat3 normalMatrix).

14. What are the cylinder equations and its caps?

The side uses x=r sinθ, z=r cosθ, and two heights y=±0.5, with r=0.5. Radial normals face outward. Each cap is a triangle fan with a separate axial normal (0,±1,0) and planar UVs, which gives a crisp boundary rather than smoothing the side into the cap.

Code: src/geometry/Primitives.cpp:139 (MeshData Cylinder).

15. What is the cone equation?

Its apex is at y=0.5, base at y=-0.5, and radius varies as ρ(y)=0.5(0.5-y). The side satisfies $x^2+z^2 = [0.5(0.5-y)]^2$. The generator connects base samples to the tip and adds a bottom cap; side normals are sloped, not merely radial.

Code: src/geometry/Primitives.cpp:189 (MeshData Cone), src/math/Ray.cpp:144 (RayHit Cone).

16. What are triangle winding and back-face culling?

Vertex order defines which side is the front. The generators use counterclockwise faces as seen from outside. Raster back-face culling avoids drawing back-facing opaque triangles. A one-sided floor viewed from below can disappear; a floor slab provides both top and underside surfaces instead.

Code: src/geometry/Primitives.cpp:13 (void addQuad), src/render/Renderer.cpp:315 (glCullFace), src/math/Ray.cpp:56 (RayHit Plane).

17. How are Woody and Jessie constructed?

Both use the Humanoid constructor with different HumanoidStyle materials and details. Hip, shoulder and head joints carry shapes for torso, limbs, boots, hands, hat and facial details. Their shared construction/animation avoids duplicating nearly identical code.

Code: src/characters/Humanoid.cpp:68 (Humanoid::Humanoid), src/characters/Humanoid.h:14 (struct HumanoidStyle). Report: 3.2.

18. What makes Bullseye and Penny different from humanoids?

They have four independent leg joints and elongated bodies assembled from ellipsoids, cylinders and cones. Bullseye also exposes a saddle-seat joint. Their movement proxies use multiple short spine boxes so a long body can turn through doorways without one oversized AABB.

Code: src/characters/Bullseye.cpp:11 (Bullseye::Bullseye), src/characters/Cat.cpp:9 (Cat::Cat), src/world/PhysicsWorld.cpp:129 (void PhysicsWorld::AppendActorBounds). Report: 3.3, 3.6, 4.8.

19. How are the house, stairs and doors created mathematically?

BuildHouse combines box and plane instances. A box with corners lo,hi has centre (lo+hi)/2 and size hi-lo. Eighteen stair boxes use rise=-Ground/StairSteps and tread=(StairTopZ-StairBottomZ)/StairSteps. Door leaves are children of hinge nodes, so hinge rotation opens them.

Code: src/world/House.cpp:29 (SceneNode* Box), src/world/House.cpp:175 (constexpr int steps), src/world/StoryProps.cpp:338 (StoryRig BuildStoryProps). Report: 3.11–3.12.

20. How do you model a sloped rail or roof without an external model?

The stair handrail length is sqrt(run²+climb²) and its tilt uses atan2(run,climb) because the cylinder's original axis is Y. Roof pieces are rotated/scaled boxes; gables use a transformed diamond with its lower half hidden in the wall. These are geometric assemblies, not arbitrary imported assets.

Code: src/world/House.cpp:186 (const float railLength), src/world/House.cpp:51 (void Gable).

21. What is tessellation, and what is the project's LOD?

Tessellation approximates a curved surface with triangles. Assets::Load creates full, medium and low sphere/cylinder/cone meshes. ForScreenSize selects detail using bounding radius divided by camera distance; thresholds are 0.012 and 0.06. Analytic ray tracing intersects the primitive equation, so it does not need this triangle LOD.

Code: src/render/Assets.cpp:14 (void Assets::Load), src/geometry/Mesh.h:42 (const Mesh& ForScreenSize). Report: 4.11.

C. Transformation, hierarchy and linear algebra

22. Where did you define translation, scale, rotation, shear and reflection?

The explicit 4×4 matrices are in Transform3D.cpp: translate, scale, rotateX/Y/Z, Rodrigues rotateAxis, shear, and reflect. Transform::Matrix efficiently composes the transform used by scene nodes; inspector controls modify its component values.

Code: src/math/Transform3D.cpp:24 (glm::mat4 translate), src/math/Transform3D.cpp:87 (glm::mat4 shear), src/math/Transform3D.cpp:95 (glm::mat4 reflect), src/scene/Transform.h:27 (glm::mat4 Matrix). Report: 2.2.

23. Why use homogeneous coordinates?

A point uses (x,y,z,1) and a direction uses (x,y,z,0). Translation can then join rotation and scale in a single 4×4 matrix. Translating a direction must have no effect, which is why inverse-transforming a ray direction uses W=0.

Code: src/math/Transform3D.cpp:24 (glm::mat4 translate), src/math/Ray.cpp:177 (RayHit Object).

24. What is the transformation order, and why does it matter?

M = T × Ry × Rx × Rz × B × S, applied right to left to column vectors. A shape scales in its local space, receives the extra basis transform, rotates, then translates. Matrix multiplication is not commutative: rotating then translating gives a different result from translating then rotating.

Code: src/scene/Transform.h:11 (M = T(position)), src/scene/Transform.h:27 (glm::mat4 Matrix).

25. Does GLM use row-major or column-major indexing here?

GLM's m[column][row] indexing is used. The fromRows helper accepts readable row entries but assigns the columns correctly. Confusing those conventions transposes a matrix and can rotate, project or translate incorrectly.

Code: src/math/Transform3D.cpp:8 (glm::mat4 fromRows), src/scene/Transform.h:33 (glm stores columns).

26. Explain Rodrigues' arbitrary-axis rotation.

For a normalised axis u, $R = \cos\theta\, I + (1-\cos\theta)uu^{T} + \sin\theta[u]\times$. The skew matrix represents the cross product. It rotates around any axis, rather than only X/Y/Z; rolling and inspector operations can use it.

Code: src/math/Transform3D.cpp:76 (glm::mat4 rotateAxis).

27. How do you rotate or scale about a point other than the origin?

Translate that point to the origin, apply the operation, then translate back: T(p) R T(-p) or T(p) S T(-p). A hinge node achieves a similar effect through the hierarchy because its children are positioned relative to the hinge.

Code: src/math/Transform3D.cpp:105 (glm::mat4 scaleAboutPoint), src/math/Transform3D.cpp:111 (glm::mat4 rotateAboutAxis).

28. What do shear and reflection mean?

Shear adds coordinate-dependent displacement, for example x'=x+sxy y, tilting a shape. Reflection about a plane through the origin with normal n uses $I-2nn^{T}$. The inspector stores these extra linear operations in basis. A zero scale is singular and cannot safely be inverted.

Code: src/math/Transform3D.cpp:87 (glm::mat4 shear), src/math/Transform3D.cpp:95 (glm::mat4 reflect), src/app/ToyRoomApp.cpp:1123 (void ToyRoomApp::HandleEditMode).

29. Why is the normal matrix the inverse transpose?

Tangents transform by the model's linear part A. To preserve n·t=0, the normal must transform by $(A□^{-1})^{T}$, then be normalised. Simply applying A gives wrong highlights after nonuniform scaling or shear. Translation is excluded by using the 3×3 part.

Code: src/math/Transform3D.cpp:148 (glm::mat3 normalMatrix), shaders/lit.vert:12 (uNormalMatrix). Report: 2.3.

30. Explain mounting Jessie onto Bullseye without a new movement system.

Save Jessie's world transform, detach her root, attach it beneath Bullseye's seat, and convert placement with local = inverse(seatWorld) × oldWorld. Then the existing transition moves her to the seated pose. Her world position now follows Bullseye automatically. Her independent collision actor is disabled while mounted.

Code: src/app/ToyRoomApp.cpp:1200 (void ToyRoomApp::Mount), src/scene/SceneNode.cpp:26 (void SceneNode::AttachChild). Report: 3.3.

31. How does dismounting avoid an abrupt world-space jump?

Read Jessie's seated world transform before detaching, attach her back to the world root, restore world position/heading as local values, then animate to a floor-supported spot beside Bullseye. The current code preserves position and heading for the ordinary mounting setup; it does not implement arbitrary full matrix decomposition of every possible edited shear.

Code: src/app/ToyRoomApp.cpp:1224 (void ToyRoomApp::Dismount), src/characters/Character.cpp:151 (void Character::StartTransition).

D. Camera, projection, visibility and picking

32. Where are the view and projection equations?

t3d::lookAt, perspective and orthographic define the matrices. Camera::View/Projection call them. Vertex shaders evaluate clip = projection × view × model × vertex; OpenGL performs the homogeneous divide and rasterisation.

Code: src/math/Transform3D.cpp:118 (glm::mat4 lookAt), src/math/Transform3D.cpp:130 (glm::mat4 perspective), src/math/Transform3D.cpp:140 (glm::mat4 orthographic), shaders/lit.vert:24 (gl_Position). Report: 2.4.

33. Derive the camera basis in lookAt.

f=normalize(target-eye), r=normalize(f×up), u=r×f. View rows are r, u and -f, with translation entries -r·eye, -u·eye, f·eye. This expresses world positions relative to the eye in the camera's coordinate system.

Code: src/math/Transform3D.cpp:118 (glm::mat4 lookAt).

34. What is perspective projection and its FOV/aspect relation?

With t=tan(fovY/2), the X/Y projection factors are 1/(aspect*t) and 1/t. The output W is -viewZ, so farther objects shrink after dividing by W. Near/far determine depth mapping; they are clipping distances, not focal length controls.

Code: src/math/Transform3D.cpp:130 (glm::mat4 perspective), src/scene/Camera.h:21 (nearPlane).

35. How does orthographic projection differ?

It linearly maps a box into clip space without perspective shrinking; W remains 1. The current renderer uses orthographic projection for the sun/moon shadow camera. The normal gameplay camera uses perspective even when a front/side/top orientation preset is chosen.

Code: src/math/Transform3D.cpp:140 (glm::mat4 orthographic), src/render/Renderer.cpp:147 (void Renderer::RenderSunShadow).

36. What are the three camera modes?

Free lets the eye move independently; Orbit positions it around a target; Follow tracks the selected character and smooths its view. C cycles modes and F focuses/orbits. Orbit spherical offsets use sine/cosine of yaw and pitch. Story shots set this same camera's eye and target.

Code: src/scene/Camera.cpp:34 (void Camera::ApplyOrbit), src/app/ToyRoomApp.cpp:839 (void ToyRoomApp::HandleCamera), src/world/StoryDirector.cpp:330 (bool StoryDirector::Cinematic).

37. How is a mouse click converted into an object selection?

ViewRay converts cursor position to a world ray using the camera basis, FOV and aspect. Pick transforms it into each visible primitive's object space and selects the smallest positive hit distance. ownerId maps a hit limb/part back to its selectable parent. Nonselectable geometry can occlude a selectable object.

Code: src/app/ToyRoomApp.cpp:796 (Ray ToyRoomApp::ViewRay), src/app/ToyRoomApp.cpp:812 (int ToyRoomApp::Pick), src/scene/SceneNode.cpp:82 (void SceneNode::SetOwner).

38. Why is mouse picking different from clue inspection?

Picking chooses an object by a ray under the cursor. ENTER inspection checks Penny's XZ distance to physical clue positions, so the player need not click a tiny clock pixel. Both use the existing Input and application handlers; there is no separate puzzle input library.

Code: src/app/ToyRoomApp.cpp:668 (void ToyRoomApp::HandleSelection), src/world/HallwayPuzzle.cpp:212 (std::string HallwayPuzzle::Interact), src/core/Input.h:15 (class Input).

39. What are frustum culling and conservative bounds?

Frustum culling skips objects entirely outside the six view-volume planes. A bound is conservative when it contains every transformed point; it may include extra empty space but must not omit geometry. unitBoundsRadius checks transformed corners so shear does not cause objects to disappear incorrectly.

Code: src/render/Renderer.cpp:80 (struct Frustum), src/math/Transform3D.cpp:153 (float unitBoundsRadius). Report: 2.9.

40. How does camera collision work, including the floor fix?

MoveCamera sweeps a small box through existing scenery using the same collision solver. CameraSightline shortens the target-to-eye segment before obstructions; the app restores distance gradually. Floor slabs and stair cubes now carry cameraSolid, with camera-only bounds so they stop the camera while actors still climb through the existing floor-support logic.

Code: src/world/PhysicsWorld.cpp:380 (glm::vec3 PhysicsWorld::MoveCamera), src/world/PhysicsWorld.cpp:385 (glm::vec3 PhysicsWorld::CameraSightline), src/scene/SceneNode.h:35 (bool cameraSolid), src/app/ToyRoomApp.cpp:435 (glm::vec3 sight = physics.CameraSightline). Report: 4.8.

E. Illumination and shading

41. Where is the illumination equation actually implemented?

shaders/lighting.glsl is shared by raster and ray shaders. The model combines ka Ia C + Σ visibility*attenuation*intensity*lightColour*(kd max(N·L,0) C + ks specular) + emissive. C is material colour multiplied by texture colour. This is local illumination, not a physically complete light-transport simulation.

Code: shaders/lighting.glsl:116 (void addLight), shaders/lighting.glsl:140 (vec3 ambientTerm), shaders/lit.frag:53 (vec3 color = albedo). Report: 2.5.

42. Define ambient, diffuse, specular and emissive.

Ambient approximates unresolved background illumination. Diffuse gives matte orientation-dependent response. Specular gives view-dependent highlights. Emissive adds the object's own apparent colour; it does not automatically create an illuminating light. Bulbs/laser glows therefore also have separate light slots.

Code: src/scene/Material.h:16 (struct Material), shaders/lighting.glsl:140 (vec3 ambientTerm), src/app/ToyRoomApp.cpp:1262 (void ToyRoomApp::UpdateLights).

43. Why use max(N dot L, 0)?

Lambert's cosine law makes brightness proportional to the cosine between normal N and direction-to-light L. Both vectors are normalised. Negative dot products mean the light is behind the surface, so clamping to zero prevents negative illumination.

Code: shaders/lighting.glsl:123 (float NdotL).

44. How many light types and actual sources are there?

Three types: directional, point and spot. There are eight configured slots: one sky directional; bulb point; lamp spot; two headlight spots; laser point; area point; ghost/story point. The sky slot represents sun or moon as time changes, and the area/glow slots are reused by location/story. Eight slots does not mean eight always-enabled lights.

Code: src/scene/Light.h:8 (enum class LightType), src/app/ToyRoomApp.cpp:29 (SkyLight), src/app/ToyRoomApp.cpp:1262 (void ToyRoomApp::UpdateLights). Report: 4.1.

45. Explain directional, point and spot vectors.

Directional light uses parallel rays and L=normalize(-direction) without distance falloff. A point light uses L=normalize(lightPosition-P). A spot begins with the point-light vector and restricts illumination to a cone about its travel direction.

Code: shaders/lighting.glsl:93 (void lightVector).

46. What is attenuation?

Point/spot strength scales by 1/(kc+kl*d+kq*d²). d is distance to the light; coefficients control falloff. Directional light uses factor 1. The shader guards division of the light vector near zero distance; coefficients come from each CPU-side Light.

Code: src/scene/Light.h:25 (float constant), shaders/lighting.glsl:105 (attenuation = 1.0 /).

47. How is the spotlight edge softened?

theta=dot(-L,normalize(direction)) measures the cosine of the spot angle. smoothstep(outerCutoff,innerCutoff,theta) fades from dark outside the outer cone to full strength inside the inner cone. Stored cutoffs are cosines of half-angles, not raw degrees.

Code: shaders/lighting.glsl:108 (float theta), src/scene/Light.h:30 (float innerCutoff).

48. Explain Phong specular reflection.

R=reflect(-L,N) is the reflected incoming light direction. Specular response is ks*max(R·V,0)^n, where V points to the eye and n is shininess. Higher n gives a narrower highlight; ks controls strength. The response is evaluated only on the light-facing side.

Code: shaders/lighting.glsl:133 (vec3 R = reflect).

49. Explain Blinn-Phong and the exponent used in this project.

Blinn uses the halfway vector H=normalize(L+V) and max(N·H,0). This shader uses exponent 4*shininess to make highlights roughly comparable to its Phong choice. That factor is this implementation's approximation, not a universal identity for all surfaces and angles.

Code: shaders/lighting.glsl:130 (vec3 H = normalize).

50. How many shading modes, and what changes between them?

Flat uses a constant face normal per triangle. Gouraud evaluates illumination at vertices and interpolates the results. Phong interpolates normals and evaluates illumination per fragment with reflection-vector specular. Blinn also evaluates per fragment but uses halfway-vector specular. F2 cycles these four raster choices.

Code: src/render/RenderSettings.h:6 (enum class ShadingMode), shaders/lit.frag:42 (vec3 N = uShadingMode), shaders/gouraud.vert:24 (void main). Report: 2.6.

51. Is Phong shading the same as Phong lighting?

No. Phong shading specifies interpolation of normals and per-fragment evaluation. Phong lighting names the reflection-vector specular formula. A per-fragment shader can use Blinn lighting instead. The project's four labels combine these interpolation/model choices for demonstration.

Code: shaders/lit.frag:51 (computeLighting), shaders/lighting.glsl:129 (if (uBlinn == 1)).

52. Why can Gouraud miss a shiny highlight?

If the highlight lies between vertices, none of the evaluated vertex colours includes its maximum. Interpolation cannot recover a peak it never sampled. Per-fragment Phong/Blinn evaluate it at interior pixels, so they capture smaller highlights better.

Code: shaders/gouraud.vert:32 (computeLighting), shaders/gouraud.frag:22 (void main). Demonstrate on a sphere at the same camera/light pose.

53. How is Flat shading produced without rebuilding all normals?

The fragment shader computes normalize(cross(dFdx(worldPosition),dFdy(worldPosition))). Those derivatives lie along a triangle's surface, so their cross product gives its face normal. This produces facets even if the mesh also stores smooth normals for the other modes.

Code: shaders/lit.frag:42 (vec3 N = uShadingMode).

54. How do material parameters distinguish cloth, brass and wood?

They use different colour, ka/kd/ks and shininess values, plus image/procedural detail. Cloth has weak broad highlights; brass has stronger sharper ones; polished wood can also reflect in ray tracing. reflectivity governs secondary mirror rays and is distinct from local specular ks.

Code: src/render/Assets.cpp:147 (Material& Assets::Mat), src/scene/Material.h:16 (struct Material), src/world/Room.cpp:51 (floorMat.reflectivity). Report: Appendix B.

55. How do lights follow animated objects?

UpdateLights reads world positions and transformed axes of lamp/headlight/laser anchors after hierarchy updates. The lamp spot follows the head's -Y axis; headlights follow the car's local +Z direction. Moving the model therefore changes actual illumination, not just a decorative bulb mesh.

Code: src/app/ToyRoomApp.cpp:1262 (void ToyRoomApp::UpdateLights), src/app/ToyRoomApp.cpp:53 (glm::vec3 AxisOf).

56. How can you prove ambient/diffuse/specular independently?

Hold camera/time/object fixed, toggle F5/F6/F7 one term at a time, then restore them. F8 toggles the sky light; lamp power/direction demonstrates local lights. Compare the same settings across renderers; changing the clock/camera at the same time makes comparison unreliable.

Code: src/app/ToyRoomApp.cpp:487 (void ToyRoomApp::HandleGlobalKeys), src/render/Renderer.cpp:242 (void Renderer::UploadLights). Report: 4.1–4.2.

F. Textures and surface detail

57. Where are texture equations and loading defined?

ProceduralTextures.cpp generates CPU RGBA images. BmpLoader.cpp loads image files into the same Image format. Texture.cpp uploads 2D images to OpenGL. Assets::Load associates texture names/slots with materials and creates the traced texture array.

Code: src/render/ProceduralTextures.cpp:131 (Image WoodFloor), src/render/BmpLoader.cpp:30 (std::optional<Image> Load), src/gl/Texture.cpp:5 (Texture::Texture), src/render/Assets.cpp:14 (void Assets::Load). Report: 2.7.

58. What are UV coordinates and UV scaling?

UVs locate a point in a 2D image. The shader samples at UV*uvScale; values beyond 0–1 repeat when wrapping is GL_REPEAT. Giving a wide wall more repeats keeps wallpaper/wood detail at a reasonable world scale instead of stretching a single pattern across it.

Code: src/geometry/Vertex.h:9 (uv), shaders/lit.frag:26 (vec4 texel), src/gl/Texture.cpp:12 (GL_TEXTURE_WRAP_S).

59. How are sphere and box textures mapped?

The sphere maps longitude to U and latitude to V, with a seam at the wrap. Cube faces each have their own planar coordinates so textures are oriented on all six sides. The analytic ray shader reconstructs the corresponding UV from the object-space hit and face normal.

Code: src/geometry/Primitives.cpp:100 (MeshData Sphere), src/geometry/Primitives.cpp:53 (MeshData Cube), shaders/raytrace.frag:182 (vec2 primitiveUV(int type, vec3 p, vec3 n)).

60. How do procedural noise and fBm work?

Hash gives deterministic pseudo-random samples; ValueNoise smoothly interpolates neighbouring samples. Fbm adds multiple noise octaves at increasing frequency and decreasing amplitude. The texture generators combine noise with lines, grids or periodic functions to produce grain, weave and aged variation.

Code: src/render/ProceduralTextures.cpp:12 (float Hash), src/render/ProceduralTextures.cpp:19 (float ValueNoise), src/render/ProceduralTextures.cpp:33 (float Fbm).

61. Are procedural textures recomputed every frame?

No. They are generated and uploaded by Assets::Load, then sampled on the GPU. Animation changes transforms/material state rather than rebuilding every image. Procedural source files remain inspectable, and the CPU images can be exported.

Code: src/render/Assets.cpp:14 (void Assets::Load), src/app/ToyRoomApp.cpp:1803 (void ToyRoomApp::ExportScene).

62. Explain the BMP reader, row padding and orientation.

The reader checks the header, dimensions, bit depth and supported compression values, calculates rows padded to a multiple of four bytes, handles positive/negative height orientation, and converts stored BGR/BGRA to RGBA. It does not provide a general JPEG/PNG decoding framework. The clock is supplied to this runtime pipeline as clock.bmp.

Code: src/render/BmpLoader.cpp:30 (std::optional<Image> Load), src/render/Assets.cpp:72 (clockPath).

63. How is the Toy Story clock textured and still clue 5?

BuildClock builds a circular rim/body and a thin cylinder face whose planar cap receives ClockSlot. The runtime BMP is loaded through the existing asset pipeline. Artwork and displayed clock numbers are appearance; the clue answer is explicitly 5 in HallwayPuzzle::Interact. Inspection uses proximity, not pixel selection.

Code: src/world/HallwayPuzzle.cpp:121 (void BuildClock), src/render/Assets.cpp:72 (clockPath), src/world/HallwayPuzzle.cpp:219 (Old Wall Clock: A faded number 5).

64. What are texture filtering and mipmaps?

Bilinear filtering interpolates nearby texels. Mipmaps store progressively smaller images to reduce distant aliasing. The default texture setup uses linear magnification and trilinear minification (GL_LINEAR_MIPMAP_LINEAR); a nearest option is also available. They improve sampling, not the underlying mesh shape.

Code: src/gl/Texture.cpp:14 (GL_TEXTURE_MIN_FILTER), src/gl/Texture.cpp:19 (glGenerateMipmap).

65. Why use a texture array for ray tracing?

An array shares one sampler across 27 surface layers and avoids exhausting separate texture units. Assets::Load bilinearly resamples each layer to 512×512. A hit's material supplies layer index and UV scale. Normalised coordinates preserve the intended mapping; resampling can still lose fine source detail.

Code: src/render/Assets.cpp:77 (constexpr int side = 512), shaders/raytrace.frag:288 (vec3 sampleSlot), src/render/Assets.h:19 (enum TextureSlot).

66. What is alpha cutout, and how does it differ from transparency?

Cutout discards texels below an alpha threshold, making hard holes such as picket gaps. Transparency blends a surface colour with what is behind it. Raster shaders discard cutouts; ray traversal skips cutout hits. The traced cutout alpha sampling explicitly uses base mip level for slot 14 to avoid unreliable ray-pixel derivatives.

Code: shaders/lit.frag:27 (if (uCutout), shaders/raytrace.frag:201 (bool cutAway), shaders/raytrace.frag:183 (float sampleSlotAlpha(int slot, vec2 uv)).

67. Does disabling textures disable materials or lighting?

No. F3 switches texture sampling; materials still provide colour and lighting coefficients. DisplayColor can supply a plain colour when a textured material has a designated preview tint. F5/F6/F7 and lighting controls are independent of texture visibility.

Code: src/scene/Material.h:20 (DisplayColor), src/render/Renderer.cpp:275 (void Renderer::ApplyMaterial).

G. Raster rendering, shadows and ray tracing

68. Explain the raster pipeline using this code.

Meshes upload indexed vertex data through VAO/VBO/EBO objects. A vertex shader transforms positions; clipping, perspective divide and rasterisation produce fragments. Fragment shaders sample textures and shade them; depth testing chooses visible surfaces. Transparent objects are handled separately with blending rather than ordinary opaque depth writes.

Code: src/geometry/Mesh.cpp:6 (Mesh::Mesh), src/render/Renderer.cpp:120 (void Renderer::Render), shaders/lit.vert:18 (void main), shaders/lit.frag:24 (void main).

69. What are VBO, EBO, VAO and FBO?

A VBO stores vertex attributes; an EBO stores triangle indices; a VAO records how those buffers are interpreted. An FBO targets off-screen attachments rather than the window. This project uses FBOs for shadow depths and the reduced-resolution ray image.

Code: src/geometry/Mesh.cpp:6 (Mesh::Mesh), src/render/Renderer.cpp:16 (void Renderer::Init), src/render/RayTracer.cpp:78 (void RayTracer::EnsureTarget).

70. How do raster shadow maps work here?

Render scene depth from the lamp or sun/moon light camera. Transform a shaded point into that light's clip space and compare its depth with the stored depth. Lamp projection is perspective; sky projection is orthographic. A 3×3 comparison neighbourhood softens the boundary. Bias reduces self-shadow acne.

Code: src/render/Renderer.cpp:120 (void Renderer::RenderLampShadow), src/render/Renderer.cpp:147 (void Renderer::RenderSunShadow), shaders/lighting.glsl:76 (float lampVisibility), shaders/lighting.glsl:62 (float sunVisibility). Report: 2.8.

71. What are shadow acne, bias and PCF?

Shadow acne is false self-shadowing caused by depth precision and surface sampling. Bias offsets the depth comparison; too much creates detached shadows. Percentage-closer filtering averages nearby depth comparison results. It softens map edges but is not a physically accurate area-light simulation.

Code: shaders/lighting.glsl:68 (float bias), shaders/lighting.glsl:73 (visibility / 9.0).

72. Where are ray-tracing equations actually written?

CPU analytic equations are in Ray.cpp for picking/laser queries and tests. GPU equivalents are in raytrace.frag for each image pixel. RayTracer.cpp constructs bounds/BVH and uploads instances; it is not where the per-pixel sphere quadratic is solved.

Code: src/math/Ray.cpp:110 (RayHit Sphere), shaders/raytrace.frag:85 (float hitSphere), src/render/RayTracer.cpp:99 (void RayTracer::Render).

73. What is a ray and how is its primary direction formed?

A ray is p(t)=origin+t*direction. The GPU uses normalize(forward + ndcX*tan(fov/2)*aspect*right + ndcY*tan(fov/2)*up) from the camera eye. Nearest positive t identifies the visible hit. The same camera basis gives picking a compatible view.

Code: src/math/Ray.h:8 (struct Ray), shaders/raytrace.frag:305 (vec3 rd = normalize).

74. Why transform the ray into object space without renormalising its direction?

Use o'=M□¹(o,1) and d'=M□¹(d,0). Then M(o'+t d')=o+t d, so the solved t remains comparable between differently scaled instances. Renormalising d' would alter t and can select the wrong closest object. Unit primitive equations then handle all instances.

Code: src/math/Ray.cpp:177 (RayHit Object), shaders/raytrace.frag:166 (float hitInstance).

75. Derive ray–sphere intersection.

Substitute o+t d into p·p=r²: a=d·d, b=2 o·d, c=o·o-r². Solve a t²+b t+c=0; negative discriminant means a miss. Choose the nearest acceptable positive root, including the exit root if the ray starts inside. This project's unit radius is 0.5.

Code: src/math/Ray.cpp:110 (RayHit Sphere), shaders/raytrace.frag:85 (float hitSphere).

76. Explain the cube slab intersection method.

Each axis gives an interval where the ray is between that axis's box faces. The intersection is [max(axisNear),min(axisFar)]; if near exceeds far, miss. If the origin is inside, use the exit. Parallel-axis handling avoids dividing by a near-zero direction on the CPU.

Code: src/math/Ray.cpp:72 (RayHit Cube), shaders/raytrace.frag:62 (float hitCube).

77. How are cylinder and cone ray hits found?

Substitute the ray into the cylinder's x²+z²=0.25 or cone's varying-radius quadratic, solve roots and reject hits outside y□[-0.5,0.5]. Test end disks separately. Degenerate quadratics become linear equations where needed. Choose the nearest surviving side/cap hit and its correct normal.

Code: src/math/Ray.cpp:125 (RayHit Cylinder), src/math/Ray.cpp:144 (RayHit Cone), src/math/Ray.cpp:13 (float SolveQuadratic), shaders/raytrace.frag:131 (float hitCone).

78. What is a BVH, and how is this one built?

A bounding volume hierarchy groups primitives beneath axis-aligned boxes. Build computes the group's bound, chooses the longest centre-spread axis, partitions around the median and recursively builds two children; leaves contain at most two objects. Rays reject entire groups whose bounds they miss. This is median splitting, not an SAH builder.

Code: src/render/RayTracer.cpp:43 (void RayTracer::Build), src/render/RayTracer.h:39 (LeafSize). Report: 2.9.

79. Is BVH ray tracing always O(log n)?

No. A balanced tree often reduces work substantially, but a ray can cross many overlapping bounds; worst-case traversal still approaches testing all objects. This project's tree is rebuilt from current draw items every frame so animation and hidden/revealed toys remain correct. A static/dynamic split or refit would be a future optimisation.

Code: src/render/RayTracer.cpp:99 (void RayTracer::Render), shaders/raytrace.frag:210 (int traceClosest).

80. How are transformed AABBs and BVH data packed?

For a unit box, world half extent is 0.5*(abs(Mcol0)+abs(Mcol1)+abs(Mcol2)), with a small pad. Instance records use eight vec4 texels for inverse transform/material; each BVH node uses two vec4s for bounds and child/leaf information. The GLSL buffer sampler fetches those records.

Code: src/render/RayTracer.cpp:99 (void RayTracer::Render), src/render/RayTracer.h:37 (TexelsPerInstance), shaders/raytrace.frag:48 (vec4 fetch(int instance).

81. How do ray-traced shadows work?

Start a secondary ray slightly above the hit surface toward each relevant shadow-casting light. A closer opaque intersection before the point light means blocked; directional light uses a far distance. The shader skips sufficiently weak contributions to save work. This produces geometry-dependent shadows, not a second shadow-map renderer.

Code: shaders/raytrace.frag:241 (bool occluded), shaders/raytrace.frag:341 (vec3 origin = P + N), shaders/raytrace.frag:349 (strength < 0.004).

82. How are reflection and transparency implemented?

An opaque reflective surface adds its local shading weighted by 1-reflectivity, scales path throughput by reflectivity and continues along reflect(direction,N). A transparent surface adds opacity-weighted local shading and continues straight through with throughput 1-opacity. At the last permitted hit, the finite path contributes its local shading and terminates.

Code: shaders/raytrace.frag:361 (float opacity = t5.a), shaders/raytrace.frag:358 (bounce == uMaxBounces).

83. Does the tracer simulate refraction or full global illumination?

No. Transparency continues straight rather than applying Snell's law. There is no Monte Carlo path sampling, diffuse interreflection or physically simulated area-light transport. It is a finite Whitted-style analytic tracer with direct illumination, selected shadows, mirror bounces and approximate transparency. Describe those implemented features precisely.

Code: src/render/RayTracer.h:16 (GPU Whitted-style ray tracer), shaders/raytrace.frag:366 (continue straight through). Report: 4.4 and Conclusions.

84. Why offset secondary ray origins and stop weak paths?

Starting exactly on a surface can repeatedly hit that surface because of floating-point error. The code uses a small offset (2e-3 world units) along the normal or transmitted direction. A bounce limit and throughput threshold keep the loop finite and avoid tracing negligible contributions.

Code: shaders/raytrace.frag:372 (ro = P + N), shaders/raytrace.frag:374 (throughput.b)) < 0.02).

85. Why can raster and ray-traced pictures differ?

Raster surfaces are triangle approximations, while tracing uses exact primitive equations. Shadow methods, transparency ordering and reflections also differ, and the default traced image is half resolution. Scene/material/lighting inputs are shared, but those algorithms do not promise pixel-identical output.

Code: src/geometry/Primitives.cpp:100 (MeshData Sphere), src/render/RayTracer.cpp:78 (void RayTracer::EnsureTarget), src/render/RenderSettings.h:38 (float rayScale).

86. Does choosing Flat or Gouraud change analytic ray geometry?

No. The four interpolation demonstrations belong to the raster path. The ray shader shades analytic normals at the hit; the settings can select its Phong/Blinn specular choice, but it does not interpolate Gouraud colours over raster triangles or compute flat tessellation normals. Use rasterisation for the four-mode comparison.

Code: shaders/raytrace.frag:324 (vec3 N = normalize), src/render/Renderer.cpp:242 (void Renderer::UploadLights), shaders/gouraud.vert:24 (void main).

H. Animation, interaction and movement

87. How many interactions does the project support?

There is no authoritative enum called “interaction type.” A useful explanation has eight families: selection/live ownership, movement, proximity story/puzzle actions, character special actions, camera controls, transform editing, render inspection, and world/time controls. Demonstrate examples in each. Target acquisition specifically has mouse-ray picking and world-space proximity; numbered keys also select known targets.

Code: src/app/ToyRoomApp.cpp:1704 (void ToyRoomApp::PrintHelp), src/app/ToyRoomApp.cpp:1023 (void ToyRoomApp::HandleObjectControl), src/world/StoryDirector.cpp:358 (std::string StoryDirector::Interact).

88. What makes movement frame-rate aware?

Position changes by velocity times dt, turning by degrees-per-second times dt, and speed approaches its requested value with bounded acceleration. Application::Run caps unusually large frame time. Physics uses fixed substeps. This improves consistency; it is not a guarantee of perfect numerical equivalence at arbitrary frame rates.

Code: src/characters/Character.cpp:36 (void Character::Drive), src/core/Application.cpp:59 (const float dt), src/world/PhysicsWorld.cpp:410 (void PhysicsWorld::Update).

89. How are walking animations calculated?

walkPhase advances with distance, and sine waves swing opposite/diagonal leg pairs. A motion blend fades gait in/out. Cat body bob, tail and eyelids use additional periodic functions; humanoids also animate arms and pose features. These are procedural joint animations rather than imported skeletal clips.

Code: src/characters/Character.cpp:78 (void Character::Animate), src/characters/Cat.cpp:97 (void Cat::Animate), src/characters/Humanoid.cpp:194 (void Humanoid::Animate), src/characters/Bullseye.cpp:101 (void Bullseye::Animate). Report: 4.6.

90. How are transitions and jumping different from ordinary walking?

StartTransition/UpdateTransition interpolate position/heading over a duration for authored hops such as chest emergence or mounting. Bullseye's jump adds JumpHeight*sin(pi*t) to the body and folds legs. Normal gameplay uses the existing controller and physics; scripted transitions are explicitly exempted from ordinary actor constraints while active.

Code: src/characters/Character.cpp:163 (void Character::UpdateTransition), src/characters/Bullseye.cpp:128 (jumpLift = JumpHeight), src/app/ToyRoomApp.cpp:361 (Mounting hops, climbing out).

91. What is Buzz's flight system?

Buzz inherits the character controller with flight enabled. Vertical input changes Y; 3D waypoints drive the story flight through the same character path. Buzz::Animate blends flight pose, wings and leg positions based on flight conditions. The wardrobe release and door approach reuse it rather than inventing a separate moving billboard.

Code: src/characters/Humanoid.cpp:248 (Buzz::Buzz), src/characters/Humanoid.cpp:338 (void Buzz::Animate), src/characters/Character.cpp:52 (if (canFly)), src/world/StoryDirector.cpp:643 (case Rescue::BuzzFlight). Report: 3.4.

92. How is the laser aimed and made to hit something?

LaserTip and LaserDirection derive the beam origin/axis from the laser joint. AimAt computes pitch toward a target. PhysicsWorld::FireLaser tests visible geometry with analytic rays and reports the nearest hit; that distance limits the rendered beam. A dynamic block receives impulse/tumbling, while the story checks a real entrance-leaf hit.

Code: src/characters/Humanoid.cpp:315 (glm::vec3 Buzz::LaserTip), src/characters/Humanoid.cpp:330 (void Buzz::AimAt), src/world/PhysicsWorld.cpp:507 (float PhysicsWorld::FireLaser), src/app/ToyRoomApp.cpp:421 (buzz->SetLaserLength).

93. How does the ball roll, and how do the car wheels turn?

Ball travel determines the rolling axis and angle; accumulated rotation is stored in the shape's basis so motion in changing directions composes correctly. Car wheel spin comes from travel and turning affects steering pose. Wheel/ball animation is visual articulation; it is not a complete tyre-friction or vehicle dynamics simulation.

Code: src/world/Environment.cpp:222 (void Environment::UpdateBall), src/characters/RCCar.cpp:78 (void RCCar::Animate). Report: 3.5, 4.6.

94. How are companions guided through the house?

StoryDirector::Init defines 13 navigation nodes and 12 links across seven zones, then runs Floyd–Warshall to precompute shortest path cost/next hop. Navigate chooses doorway waypoints between zones and Steer avoids nearby bodies. CompanionFollow assigns path-relative slots and controls following/settling distances. This is a small waypoint graph, not a generated navigation mesh.

Code: src/world/StoryDirector.cpp:58 (Doorway nodes of the navigation graph), src/world/StoryDirector.cpp:843 (bool StoryDirector::Navigate), src/world/CompanionFollow.cpp:56 (glm::vec3 FormationAnchor::Slot). Report: 4.9.

95. Why do companions not start moving whenever Penny turns?

Formation activation depends on leader translation rather than heading alone. A recorded trail determines slot positions; start/stop distances and a settled/approaching state avoid jitter from tiny taps. Releasing a manually controlled toy rejoins it only when sufficiently far from its slot. The settings specify these thresholds.

Code: src/world/CompanionFollow.cpp:26 (void FormationAnchor::Update), src/world/CompanionFollow.cpp:88 (CompanionFollower::Command CompanionFollower::Update), src/world/CompanionFollow.h:25 (struct FollowSettings).

96. Can manual character control coexist with the story?

Yes. SetControlled tells the story which actor (and mounted passenger) belongs to the player; automated tasks skip those actors while other toys continue. Releasing control returns them to following/task behaviour. Full manual sandbox is a separate explicit mode. The controller and scene objects are shared.

Code: src/world/StoryDirector.cpp:317 (void StoryDirector::SetControlled), src/app/ToyRoomApp.cpp:721 (void ToyRoomApp::Select), src/app/ToyRoomApp.cpp:761 (void ToyRoomApp::TogglePennyControl).

I. Physics, floor support and the reported bug

97. What physics is actually simulated?

Actors/cameras use conservative transformed proxies, swept AABB collision, sliding and overlap recovery. Dynamic blocks use fixed-step gravity, velocity/angular updates and contact correction. The laser can transfer impulses. It is a custom limited physics world, not a general-purpose rigid-body engine with every joint/contact feature.

Code: src/world/PhysicsWorld.cpp:203 (glm::vec3 PhysicsWorld::Move), src/world/PhysicsWorld.cpp:410 (void PhysicsWorld::Update), src/world/PhysicsWorld.cpp:507 (float PhysicsWorld::FireLaser). Report: 4.7.

98. What is an AABB, and how do you transform its extent?

An axis-aligned bounding box stores low/high world-space corners. For local half extent h and linear columns a,b,c, transformed half extent is abs(a)*hx+abs(b)*hy+abs(c)*hz. It is conservative under rotation/shear, but can contain empty corners. Long animals use chains of smaller boxes to reduce that excess.

Code: src/world/PhysicsWorld.cpp:13 (glm::vec3 Extent), src/world/PhysicsWorld.cpp:61 (PhysicsWorld::Bounds PhysicsWorld::ShapeBounds).

99. Why sweep movement rather than only test the final position?

Final-position overlap can miss a thin wall crossed in one fast frame. Sweeping finds the first hit along the displacement against an obstacle expanded by the moving half extent (Minkowski sum). The solver moves to contact and removes the component into the contact normal so the remaining movement slides along the face.

Code: src/world/PhysicsWorld.cpp:22 (bool Sweep), src/world/PhysicsWorld.cpp:298 (slide along the contacted face).

100. How are characters kept separate without lifting them vertically?

SeparateActors finds overlap between proxy pairs and pushes horizontally, respecting priority: player-owned or busy actors keep their place while followers yield. It calls the same scenery slide solver. Scripted mounted/emerging actors are handled separately to avoid fighting authored transitions.

Code: src/world/PhysicsWorld.cpp:319 (void PhysicsWorld::SeparateActors), src/app/ToyRoomApp.cpp:379 (Right of way, highest first).

101. How is stair support calculated?

Map Z into a stair cell: cell=(z-bottom)/(top-bottom)*StairSteps; height is Ground+(-Ground/StairSteps)*clamp(floor(cell)+1,1,StairSteps). Render construction and physics now share StairSteps. The whole actor footprint is supported, including segment edges, so a paw/hoof does not enter a higher tread while its root is lower.

Code: src/world/PhysicsWorld.cpp:94 (float PhysicsWorld::FloorHeight), src/world/PhysicsWorld.cpp:151 (auto support =), src/world/House.cpp:175 (constexpr int steps).

102. What caused the disappearing hallway/stair floors and overlapping legs?

There were two reproduced faults. The hallway floor was one-sided and had no camera obstacle, and tread cubes also lacked camera bounds; the follow camera could cross underneath/inside them. Separately, support used segment centres and missed a swinging paw's reach into the next riser; a regression measured about 0.238 world units of penetration. Walkable floors now have slab thickness/undersides, camera bounds use the existing sweep solver, and grounded support samples the full segment footprint. Bullseye's rear spine segment was also extended to cover a backward hoof swing. The corridor floor also reaches the entrance threshold.

Code: src/world/Room.cpp:72 ("HallFloor"), src/world/House.cpp:130 (Give walkable surfaces an underside), src/world/PhysicsWorld.cpp:151 (auto support =), tests/PhysicsChecks.cpp:275 (void CheckRenderedHouseFloors), tests/PhysicsChecks.cpp:241 (void CheckStairFloorContact).

103. Why not make every step a normal solid actor obstacle?

Ordinary solid riser boxes would block horizontal movement and conflict with the existing stair support/traversal logic. cameraSolid lets the same PhysicsWorld bounds/sweep machinery stop cameras while grounded actors continue using FloorHeight and footprint support. It does not disable collision with walls, doors, furniture, blocks or other actors.

Code: src/scene/SceneNode.h:35 (bool cameraSolid), src/world/PhysicsWorld.cpp:203 (cameraContacts), src/world/PhysicsWorld.cpp:78 (void PhysicsWorld::RefreshScenery).

104. Is floor support inverse kinematics?

No. The current fix conservatively lifts the actor's root to clear the highest supporting tread under its proxy footprint. It does not independently solve four leg joint positions to plant each foot on a different stair. Some feet can be visibly above lower treads; per-foot IK would be a separate animation improvement.

Code: src/world/PhysicsWorld.cpp:151 (auto support =), src/characters/Cat.cpp:97 (void Cat::Animate).

105. Why fixed-step physics and a substep cap?

The simulation integrates at 1/120 second for more stable contacts. It caps accumulated work at six steps so one slow frame cannot schedule an ever-growing backlog. This trades catch-up accuracy during long stalls for responsiveness. Application dt and story motion use their own frame-time handling.

Code: src/world/PhysicsWorld.cpp:413 (constexpr float step = 1.0f / 120.0f), src/core/Application.cpp:59 (const float dt).

J. Gameplay, puzzle, rescue and escape

106. Where is the central story state machine?

StoryDirector owns an eight-value GameplayState, transitions, state timers and rescue/door/toy flags. Update selects the state behaviour; Advance validates the source state for a transition and calls Enter. HallwayPuzzle owns the puzzle challenge, not a second story manager. The app wires a successful submission to the story transition.

Code: src/world/StoryDirector.h:40 (enum class GameplayState), src/world/StoryDirector.cpp:186 (bool StoryDirector::Advance), src/app/ToyRoomApp.cpp:649 (void ToyRoomApp::HandlePuzzleInteraction). Report: 3.12, 4.6.

107. Explain all current story transition conditions.

From → to	Condition in the current implementation
PROLOGUE → PUZZLE	Penny is inside the ground corridor: below Slab, Z<8, X between corridor edges
PUZZLE → TOY_CHEST	Keypad submits 257 after all three inspections; app reports PUZZLE_SOLVED
TOY_CHEST → BUZZ_ROOM	Chest pressed, all three emergence sequences land, then 2.8-second celebration interval
BUZZ_ROOM → WARDROBE	Nearby actor opens the ordinary Buzz-room door through ENTER
WARDROBE → FINAL_ESCAPE	Wardrobe opens and Buzz completes four 3D flight waypoints
FINAL_ESCAPE → MORNING	Door has physically been broken, leader crosses outside beyond HouseFront+0.4 below Slab
MORNING → FREE_EXPLORE	Morning clock transition exceeds 16 seconds and pullback timer exceeds 7.5 seconds; pullback starts after everyone is outside or a 35-second fallback

Code: src/world/StoryDirector.cpp:492 (void StoryDirector::Update), src/world/StoryDirector.cpp:550 (void StoryDirector::UpdateChest), src/world/StoryDirector.cpp:358 (std::string StoryDirector::Interact), src/world/StoryDirector.cpp:580 (void StoryDirector::UpdateRescue), src/world/StoryDirector.cpp:664 (void StoryDirector::UpdateEntrance). The ordinary input paths establish these conditions; Advance itself guards state/order rather than independently recomputing every physical condition.

108. What happens on arrival and when the main door closes?

PennyArrival provides the existing exterior establishing shot; after it, Penny is playable outside and walks in. The PROLOGUE interior-position condition advances the story and removes normal exterior access. Door hinge animation eases from its open angle to the closed angle, and a closed leaf blocks leaving. Returning through the interior corridor remains possible.

Code: src/world/PennyArrival.cpp:41 (void PennyArrival::Update), src/world/StoryDirector.cpp:215 (case GameplayState::PROLOGUE), src/world/StoryDirector.cpp:458 (void StoryDirector::AnimateProps).

109. How do physical clues communicate 257?

The train, circular artwork clock and coloured block display are scene-graph meshes/materials. Interact records Train=2, Clock=5, Blocks=7 and returns a caption. Digit discovery is explicit game data, not OCR or reading arbitrary visible clock positions. The props remain after inspection; proximity hints change from inspect to review.

Code: src/world/HallwayPuzzle.cpp:67 (void BuildTrain), src/world/HallwayPuzzle.cpp:121 (void BuildClock), src/world/HallwayPuzzle.cpp:98 (void BuildBlocks), src/world/HallwayPuzzle.cpp:212 (std::string HallwayPuzzle::Interact).

110. How are clue distance and keypad distance checked?

Clues choose the closest target within 2.25 world units in XZ; the keypad uses 1.75. Only the active puzzle routes ENTER to these interactions. Physics and door layout constrain access to the upstairs area. These are proximity tests; the current code does not additionally ray-test clue visibility or compare vertical distance.

Code: src/world/HallwayPuzzle.cpp:222 (float closestDistance = 2.25f), src/app/ToyRoomApp.cpp:649 (void ToyRoomApp::HandlePuzzleInteraction).

111. How does the keypad handle incomplete, missing-clue, wrong and correct input?

AddDigit accepts 0–9 until there are three characters and rejects changes after completion. Submit returns Incomplete for fewer than three, MissingClues if inspections are missing, Incorrect for other three-digit strings, and Correct for 257 with all clues. Missing/wrong attempts clear digits but preserve discovered clues. Backspace edits digits; Display supplies CODE: 2 _ _-style text.

Code: src/world/PuzzleCode.cpp:32 (bool PuzzleCode::AddDigit), src/world/PuzzleCode.cpp:46 (PuzzleCode::Result PuzzleCode::Submit), src/world/PuzzleCode.cpp:62 (std::string PuzzleCode::Display).

112. Does solving the code teleport Penny into the Toy Room?

No. Correct submission sets the puzzle's unlocked flag, and its update eases the two door hinges to ±90 degrees. The app advances to TOY_CHEST; Penny physically walks through the doorway. Objective text now directs her to the chest button. The current state is TOY_CHEST, not the older planned TOY_RESCUE name.

Code: src/world/HallwayPuzzle.cpp:277 (PuzzleCode::Result HallwayPuzzle::Submit), src/world/HallwayPuzzle.cpp:203 (void HallwayPuzzle::Update), src/world/StoryDirector.cpp:227 (std::string StoryDirector::Objective).

113. How are Woody, Jessie and Bullseye freed?

Penny approaches and presses the red chest button. A timer opens the lid; staggered emergence phases reveal each actor, transition it upward and then onto the floor, and celebrate after landing. Hidden toys are not selectable/physical actors before release. All three landing phases must finish before progress continues.

Code: src/world/StoryDirector.cpp:412 (void StoryDirector::PressChest), src/world/StoryDirector.cpp:550 (void StoryDirector::UpdateChest), src/world/StoryDirector.cpp:275 (bool StoryDirector::Hidden).

114. How does the player get to Buzz's room?

The rescued toys follow Penny back through the upper hall, down the original stairs and to the ordinary door on the ground corridor's left. ENTER near that door sets its open flag and advances BUZZ_ROOM→WARDROBE. The divider remains solid away from its doorway; Buzz is inside a wardrobe in this version, not behind an energy barrier.

Code: src/world/StoryDirector.cpp:358 (std::string StoryDirector::Interact), src/world/StoryProps.cpp:338 (StoryRig BuildStoryProps), src/world/PhysicsWorld.cpp:223 (The ground floor is a union of boxes).

115. Why are Jessie and Bullseye needed for the wardrobe?

Penny cannot reach the knobs. Trying them requests the rescue; Jessie walks to Bullseye and uses the existing mount hook, he rides into position, aligns and jumps, and Jessie reaches. Being mounted, close enough (under 1.4 XZ units) and having jump lift over 0.45 opens it. Player control can perform the same mount/ride/jump actions; automation waits while the player owns that team.

Code: src/world/StoryDirector.cpp:580 (void StoryDirector::UpdateRescue), src/app/ToyRoomApp.cpp:1200 (void ToyRoomApp::Mount), src/characters/Bullseye.cpp:128 (jumpLift = JumpHeight).

116. How does Buzz come out and demonstrate flight?

Once doors open, Buzz becomes visible and follows four 3D points: lift within the wardrobe, fly out over the room, cross the rug and land. The existing FollowWaypoint and Buzz::Animate supply travel/pose. Completion sets buzzFreed and advances to FINAL_ESCAPE.

Code: src/world/StoryDirector.cpp:643 (case Rescue::BuzzFlight), src/characters/Character.cpp:118 (bool Character::FollowWaypoint), src/characters/Humanoid.cpp:338 (void Buzz::Animate).

117. What prevents a fake laser escape that opens the door on a timer alone?

The story accumulates firing time only while buzz->LaserOn() and physics->LastLaserHit()==doorLeaf. Losing the door hit resets that timer. Over 0.65 seconds of real nearest-door impact invokes BreakDoor. Buzz flying into position alone does not open it; a nearer obstruction prevents the required hit.

Code: src/world/StoryDirector.cpp:664 (void StoryDirector::UpdateEntrance), src/world/PhysicsWorld.cpp:507 (float PhysicsWorld::FireLaser), src/world/StoryDirector.cpp:404 (bool StoryDirector::RequestLaser).

118. How is the entrance destroyed, and is it a fracture simulation?

BreakDoor hides the door leaf/lock, enables exterior passage, reveals prebuilt boards and applies impulses/angular motion to debris. Existing block physics moves those boards. It is an authored destructible effect driven by a real laser hit, not arbitrary mesh fracture or cutting geometry at runtime.

Code: src/world/StoryDirector.cpp:440 (void StoryDirector::BreakDoor), src/world/StoryDirector.cpp:95 (void StoryDirector::Build), src/world/PhysicsWorld.cpp:401 (void PhysicsWorld::PushBlock).

119. What is the actual ending state?

The characters exit, the clock blends from night through midnight to morning, the existing camera pulls back, and the state becomes FREE_EXPLORE. Escaped() is true in MORNING/FREE_EXPLORE. There is no terminal WIN enum in this revision; do not claim the older requested nighttime WIN sequence is the currently implemented ending.

Code: src/world/StoryDirector.h:82 (bool Escaped), src/world/StoryDirector.cpp:221 (case GameplayState::MORNING), src/world/StoryDirector.cpp:330 (bool StoryDirector::Cinematic).

120. Is a chasing ghost required to win in this implementation?

No. Environment supports optional haunting/ghost motion and lamp flicker, enabled with the haunt setting or suitable demonstrations. The current story's escape condition is the laser-broken door followed by leaving the house. There is no chase-loss branch in the central state enum.

Code: src/world/Environment.cpp:256 (void Environment::UpdateGhost), src/app/ToyRoomApp.cpp:1609 (environment.hauntingEnabled = launch.haunt), src/world/StoryDirector.h:40 (enum class GameplayState).

K. Day/night, interface, validation and likely follow-ups

121. Where is the day/night equation?

UpdateSky maps hour to angle a=(skyHour-6)/12*pi. Sun height is sin(a); the moon is opposite. Smooth transitions blend daylight, sky colour, ambient and light intensity. Indoor sun/moon meshes follow the window ellipse; the outdoor shader renders a directional sky dome instead of a nearby sky object.

Code: src/world/Environment.cpp:46 (void Environment::UpdateSky), shaders/sky.glsl:3 (skyRadiance). Report: 4.5.

122. How does a manual day/night override avoid restarting the story?

It changes the visual sky target and blends skyHour, while the story's state/hour continues. The app applies the daylight rule and FreezeToys: toys hold still in daylight, Penny remains available, and saved tasks resume at night. Switching appearance does not clear clue flags or replay the prologue.

Code: src/world/Environment.cpp:46 (void Environment::UpdateSkyHour), src/app/ToyRoomApp.cpp:631 (void ToyRoomApp::ToggleDayNight), src/world/StoryDirector.cpp:308 (void StoryDirector::FreezeToys). Report: 4.10.

123. How are the HUD and font rendered?

The existing Windows GDI code generates a glyph atlas. HUD quads sample it using a small OpenGL shader and blend over the final 3D frame. DrawHud provides objective, code, clue progress and controls. This uses the project's HUD/input architecture, not a third-party GUI toolkit.

Code: src/render/Hud.cpp:26 (Hud::Font Hud::MakeFont), src/render/Hud.cpp:130 (void Hud::Render), src/app/ToyRoomApp.cpp:1334 (void ToyRoomApp::DrawHud), src/app/ToyRoomApp.cpp:1417 (void ToyRoomApp::OnRender).

124. What is fog, and is it the same as a shadow?

Fog blends surface colour toward the environment's horizon colour with distance, reducing distant contrast. A shadow means a light is occluded. The raster and ray shaders share the fog function; a ray miss sees sky radiance, which is also visible in reflections/transparency.

Code: shaders/fog.glsl:4 (applyFog), shaders/raytrace.frag:317 (if (hit < 0)), src/world/Environment.cpp:103 (glm::vec3 Environment::HorizonColor).

125. Which optimisations can you defend with code?

Shared primitive meshes/texture arrays, lower mesh detail for small objects, raster frustum culling, an analytic BVH, reduced ray target resolution, cached uniform locations, reused packing vectors, an expanded transform formula and capped physics substeps. Each reduces a specific resource or operation; do not promise a fixed FPS gain without measuring on the evaluation machine.

Code: src/geometry/Mesh.h:42 (const Mesh& ForScreenSize), src/render/Renderer.cpp:80 (struct Frustum), src/render/RayTracer.cpp:43 (void RayTracer::Build), src/scene/Transform.h:27 (glm::mat4 Matrix), src/world/PhysicsWorld.cpp:414 (At most 6 steps per frame). Report: 4.11.

126. How would you explain the floor-fix regression tests?

CheckStairFloorContact drives Penny's configured spine uphill/downhill at multiple headings and checks the reach of front/rear paws against tread support. CheckToyStairFeet checks humanoid boot and Bullseye hoof reach in both headings. CheckRenderedHouseFloors builds the actual house/room, checks camera sightline/sweep obstructions, all 18 camera tread bounds, floor top/underside analytic hits, and entrance-floor continuity. They reproduce both missing camera collision and insufficient support before the fix.

Code: tests/PhysicsChecks.cpp:241 (void CheckStairFloorContact), tests/PhysicsChecks.cpp:275 (void CheckRenderedHouseFloors), tools/check-physics.ps1:1 ($ErrorActionPreference).

127. What does the full-story rendering check verify, and what does it not prove?

It launches the game through the deterministic rehearsal using normal character movement and interaction logic, rejects wrong code, accepts 257, waits for real rescues/laser impact and checks all five actual character positions outside in FREE_EXPLORE. It runs Flat/Gouraud/Phong/Blinn, ray tracing and Debug. It is automated game execution; screenshot inspection is separate, and it is not a claim that a human manually played every camera angle.

Code: tools/check_escape_rendering.py:11 (def main), src/app/ToyRoomApp.cpp:1849 (void ToyRoomApp::DemoMission). Report: 4.12.

128. How would you present evidence rather than say “it compiles”?

Show Debug/Release build output, the physics regression output, the full escape check, and actual screenshots from running both rendering paths. Then demonstrate walking the stairs, inspecting clues, entering a wrong/correct code and firing at the door. Compilation proves syntax/linking; tests and visual inspection address behaviour and appearance.

Code: tools/build.ps1:1 (param), tools/check-physics.ps1:1 ($ErrorActionPreference), tools/check_escape_rendering.py:11 (def main).

129. What limitations should you answer honestly?

Primitive assemblies approximate character anatomy; actor/block proxies are conservative; root-based stair support is not per-foot IK; ray transparency is not refractive optics; tracing is not full global illumination; shadow-map bias/filtering are approximations; navigation uses an authored small graph; destruction uses authored debris. These are deliberate scope choices and possible future work, not features to claim as complete.

Code: src/world/PhysicsWorld.cpp:151 (auto support =), shaders/raytrace.frag:366 (continue straight through), src/world/StoryDirector.cpp:440 (void StoryDirector::BreakDoor). Report: Conclusions.

130. If the teacher asks “show me the equation,” what is your answering method?

Name the concept, write its equation, identify every variable, open the cited function, and point to the corresponding expression. Then show its visible effect with one controlled demonstration. For sphere generation open Primitives::Sphere; for Lambert/Phong open lighting.glsl; for ray quadratics open Ray.cpp and raytrace.frag; for puzzle logic open PuzzleCode::Submit; for state transitions open StoryDirector::Advance/Update.

Code: src/geometry/Primitives.cpp:100 (MeshData Sphere), shaders/lighting.glsl:116 (void addLight), src/math/Ray.cpp:110 (RayHit Sphere), src/world/PuzzleCode.cpp:46 (PuzzleCode::Result PuzzleCode::Submit), src/world/StoryDirector.cpp:186 (bool

StoryDirector::Advance).

Short rehearsal plan

1. Practise the one-minute overview and the quick-facts table.
2. Open geometry, transform, illumination and ray sources before the demonstration.
3. Show one curved object with F2, then texture and illumination-term toggles.
4. Explain mounting with the inverse-parent equation and show it.
5. Walk Penny from entrance through stairs, inspect the three clues, try a wrong code, then 257.
6. Explain chest phases, wardrobe teamwork, genuine Buzz flight and the nearest-hit door condition.
7. Show morning/free exploration and manual day/night behaviour.
8. Finish with regression evidence and the implementation limits above.

Use H for the game's on-screen guide and PrintHelp for its complete control list. In the keypad, digit keys are code entry; outside it they can select objects. State/input context determines their meaning.